Vivekanand Education Society's Institute of Technology

Department of Computer Engineering

Group No.: 19
Date : 10-02-2026

Capstone Project - I Synopsis (2025-26) - Sem VI

Large Action Model (LAM) Engine: Dione

Mrs. Geocey Shejy

Professor, Computer Engineering

Jass Suraj Shivnani V.E.S.I.T 2023.jass.shivnani@ves.ac.in	Riya Sanjay Khialani V.E.S.I.T 2023.riya.khialani@ves.ac.in
Khushi Sadhuramani V.E.S.I.T 2023.khushi.sadhuramani@ves.ac.in	Ruchika Dingria V.E.S.I.T d2023.ruchika.dingria@ves.ac.in

Table of Contents

Sr. No.:	Table of Contents	Page No.
1	Abstract	1
2	Introduction	2
3	Problem Statement	3
4	Proposed Solution	4
5	Methodology and Block Diagram	5
6	Hardware, Software and tools requirements	9
7	Proposed Evaluation Measures	10
8	Conclusion	11
9	References	12

ABSTRACT

The contemporary landscape of Artificial Intelligence has been revolutionized by Large Language Models (LLMs) that demonstrate unprecedented capabilities in generative tasks—producing human-like text, synthesizing code, and performing complex analysis. However, despite their reasoning prowess, these models function primarily as "brains in a jar," isolated from the user's immediate digital environment. They are trapped within chat interfaces, capable of suggesting actions but incapable of executing them. Current attempts to bridge this "actuation gap" typically rely on cloud-hosted agent frameworks. These solutions, while functional, necessitate a dangerous trade-off: to gain an AI assistant that can manage your life, you must expose your most sensitive personal data—private communications, financial ledgers, authentication tokens, and calendar details—to third-party servers, creating significant privacy vulnerabilities and improved attack surfaces for malicious actors.

Dione AI proposes a novel, privacy-first paradigm to resolve this dilemma. It is not merely another chatbot or a fine-tuned model, but a **Local Large Action Model (LAM) Orchestration Engine**—a robust architectural layer designed to run entirely on the user's local hardware. Dione acts as the digital nervous system for a local LLM, translating the model's unstructured, probabilistic reasoning into structured, deterministic commands that interact directly with personal APIs, desktop applications, and system utilities. By keeping the execution layer local, Dione ensures that sensitive action data never leaves the user's device, maintaining strict data sovereignty.

The system is architected around a high-performance, modular plugin interface that effectively decouples the reasoning engine (the LLM) from the execution layer (the tools). This modularity allows Dione to interface dynamically with a diverse array of communication and productivity platforms—such as WhatsApp, iMessage, Gmail, and local file systems—without relying on brittle, hard-coded conversational flows. By standardizing the communication protocol between the AI model and these external plugins, Dione enables the AI to autonomously "read" incoming message contexts, "decide" on appropriate actions based on user intent, and "execute" complex multi-step workflows. For instance, Dione can parse a natural language request to "find the meeting link sent by Alice yesterday and add it to my calendar," autonomously querying the message database, extracting the relevant information, and invoking the calendar API—all without human intervention.

This project demonstrates that **Agentic AI**—artificial intelligence capable of taking independent, meaningful action—can be achieved without the privacy compromises inherent in cloud-based systems. Dione AI serves as the foundational infrastructure for the next generation of personal computing: transforming the passive chatbot into an active, secure, and highly capable digital agent that manages the user's digital life through secure, local execution.

INTRODUCTION

The field of Artificial Intelligence has witnessed a seismic paradigm shift, transitioning from rigid, rule-based heuristics to fluid, probabilistic reasoning engines. For the past decade, the landscape of personal assistants—exemplified by early iterations of Siri, Alexa, and Google Assistant—was dominated by "command-and-control" architectures. These systems were less "intelligent" and more akin to sophisticated voice-triggered search engines, relying on brittle intent-classification pipelines. A user's request had to map perfectly to a pre-programmed script or specific syntax; if a user deviated even slightly from the expected phrasing (e.g., asking "Can you illuminate the room?" instead of "Turn on the lights"), the system would fail, often defaulting to a generic web search. This lack of semantic flexibility rendered these assistants useful only for trivial, single-shot tasks like setting timers, checking the weather, or playing music, creating a "utility ceiling" that no amount of additional hard-coded features could break.

The advent of Transformer-based architectures and the subsequent explosion of Large Language Models (LLMs) like GPT-4, Claude, and Llama 3 ushered in the era of "Generative AI." These models possess a profound semantic understanding of language, context, subtext, and logic, allowing them to engage in nuanced dialogue and perform complex reasoning tasks that mimic human cognition. However, despite this massive cognitive leap, a significant functional limitation remains: these models are fundamentally passive "Oracles." They exist in a vacuum, hermetically sealed within chat interfaces and separated from the operating system, file structures, and applications that users rely on daily. An LLM can flawlessly draft a resignation letter, generate the precise SQL query to find a customer in a database, or plan a detailed travel itinerary to Paris, but it cannot send the email, execute the query against a server, or book the flight tickets. It has the intelligence to plan, but lacks the hands to execute. This critical disconnect between cognitive capability and digital interaction is known as the "Actuation Gap"—the frustrating reality where the AI knows exactly what to do, but the user must still manually perform the labor.

To bridge this gap, the industry is currently pivoting toward "Agentic AI"—systems designed not just to converse, but to operate. Unlike a passive chatbot, an "Agent" is defined by its agency: the ability to perceive its digital environment, reason about the necessary steps to achieve a goal, and autonomously manipulate tools to execute those steps. This moves the interaction model from a simple "Request-Response" loop to a complex "Goal-Execution" loop. In this paradigm, the AI acts as a reasoning engine that drives software: it interacts with APIs, manages local file systems, controls software interfaces, and reads screen states. For example, an agentic system doesn't just tell you how to unsubscribe from a newsletter; it logs into your email, identifies the subscription link, clicks it, and confirms the unsubscription. It transforms the AI from a knowledgeable consultant into a capable worker, effectively giving the "brain in a jar" a pair of hands. This transition from "Chat" to "Action" represents the next major frontier in personal computing, promising to automate not just information retrieval, but the actual administrative labor of digital life.

PROBLEM STATEMENT

Current implementations of Agentic AI are predominantly cloud-native. Frameworks such as OpenAI's "GPTs" or Anthropic's "Computer Use" rely on a centralized architecture where user requests and data are transmitted to remote servers for inference. While this approach leverages the immense compute power of the cloud, it introduces severe architectural flaws for a personal assistant designed to manage private life. The "Convenience vs. Privacy" trade-off in this model is not merely a policy choice but a fundamental structural weakness:

1. **The Plaintext Processing Requirement (The "Black Box" Paradox):** For a cloud-based LLM to reason about a user's private messages, analyze financial PDFs, or schedule appointments, that data must be decrypted and processed in plaintext on the provider's server. Homomorphic encryption (processing encrypted data without decrypting it) is not yet viable for LLMs. This means that end-to-end encryption is effectively broken at the processing node. Regardless of a vendor's "privacy promises," the technical reality is that during inference, a copy of the user's most intimate data exists in the server's RAM, exposed to potential memory dumps, insider threats, or government subpoenas.
2. **Centralized Honeypots and Identity Hijacking:** To enable meaningful "action," users are required to store high-privilege credentials—such as Gmail OAuth tokens, banking session keys, or Slack API secrets—within the AI provider's infrastructure. This transforms the cloud provider into a massive, centralized "honeypot." A breach in this environment differs catastrophically from a standard data leak; it is not just a leak of historical logs, but a leak of *agency*. Attackers stealing these tokens do not just read the user's emails; they can actively impersonate the user, sending fraudulent messages, deleting files, or transferring funds. The risk profile shifts from "Data Breach" to "Digital Identity Hijacking."
3. **Data Sovereignty, Compliance, and the "Digital Twin":** As users interact with cloud agents over time, the provider builds a massive context window—effectively a "Digital Twin" of the user's psychology, habits, and network. Users lose ownership of this metadata. In regulated sectors (healthcare, legal, defense), sending client data to a general-purpose public cloud model for processing often violates strict compliance standards like HIPAA, GDPR, or SOC2, rendering these powerful tools unusable for professionals who need them most.
4. **Latency, Reliability, and Operational Fragility:** Cloud agents introduce unavoidable network latency, making them sluggish for real-time desktop control or voice interaction. The "round-trip" time for a simple command (User -> Cloud -> Inference -> Cloud -> User -> Action) creates a disjointed user experience compared to native applications. Furthermore, a reliance on internet connectivity renders the "personal" assistant useless in offline scenarios, creating a critical point of failure where a severed fiber cable or a cloud outage leaves the user unable to access their own automated workflows.
5. **Alignment Mismatch and Vendor Lock-in:** Cloud models are "aligned" via Reinforcement Learning from Human Feedback (RLHF) to serve a generic, mass-market safety profile. They often refuse valid, legal commands (e.g., "analyze this network packet capture") due to over-sensitive safety filters designed to protect the vendor from liability rather than to serve the user. A cloud agent is ultimately an employee of the vendor, not the user, leading to censorship of legitimate use cases and potential service termination without recourse.

PROPOSED SOLUTION

Dione AI fundamentally resolves these architectural vulnerabilities by inverting the traditional data-flow paradigm. In the current cloud-centric model, users are forced to upload their private context to a centralized intelligence for processing—a method akin to shipping your entire filing cabinet to a remote office just to have a single document summarized. Dione rejects this model. Instead, it brings the intelligence to the data. By deploying the orchestration engine directly on the user's local device, Dione eliminates the need for data transmission, ensuring that the "reasoning" occurs in the same physical location as the "storage." This shift from "Remote Intelligence" to "Edge Intelligence" is the cornerstone of the Dione architecture.

Functionally, Dione operates as a **Local Large Action Model (LAM) Orchestration Engine**. It is crucial to distinguish Dione from the Large Language Model itself; Dione is not the brain, but the nervous system. It acts as the intelligent middleware—the "connective tissue"—that sits between the raw cognitive potential of a local LLM (such as Mistral, Llama 3, or Gems) and the rigid, deterministic world of software applications. While an LLM outputs probabilities and text tokens, Dione translates these abstractions into executable API calls, file operations, and system commands. It bridges the gap between the messy, unstructured nature of human language and the structured, binary requirements of computer code.

The core proposition of Dione is to provide a standardized, secure runtime environment built upon three immutable pillars:

- **1. Zero-Trust Local Inference:** All cognitive processing and reasoning are performed strictly on-device. By leveraging modern quantization techniques (GGUF, EXL2), Dione allows consumer-grade hardware to run powerful 7-billion and 13-billion parameter models with surprisingly low latency. This ensures that the "thought process"—the analysis of your emails, the drafting of your replies—never leaves your RAM, rendering it invisible to ISPs, cloud providers, and external surveillance.
- **2. Sandboxed Plugin Execution:** Security in an agentic system is paramount; an agent that can read emails must not be allowed to unintentionally delete system files. Dione implements a strict "Sandboxed Execution Environment" for all plugins. Whether it is a WhatsApp Reader or a Calendar Writer, each tool runs in an isolated container with least-privilege access. The agent "asks" permission to use a tool, and the engine grants it only for the specific scope required, preventing a "rogue agent" scenario where a hallucinating model executes destructive commands.
- **3. Absolute Data Sovereignty:** The system adheres to a strict "No-Egress" policy for inference data. The user's interaction history, personal context, and vector database remain encrypted on the local disk. The only network traffic generated by Dione is the specific, intentional API call required to execute a task (e.g., sending the final payload to the Gmail SMTP server). The reasoning leading up to that action—the *why* and the *how*—remains exclusively within the user's control.

METHODOLOGY

The development of Dione AI follows a modular, micro-kernel-like architecture pattern, prioritizing separation of concerns between the cognitive layer (Reasoning) and the active layer (Execution). The system is divided into three primary functional planes: The User Interface (Input/Output), The Orchestration Kernel (Logic & State), and The Plugin Interface (Actuation).

5.1 System Architecture

The following diagrams illustrate the core architectural components and data flow of the Dione engine.

Diagram 1: High-Level Layered Architecture

This diagram illustrates the separation of concerns between the User Interface, the Reasoning Kernel (Brain), and the Tool Execution Layer (Body).

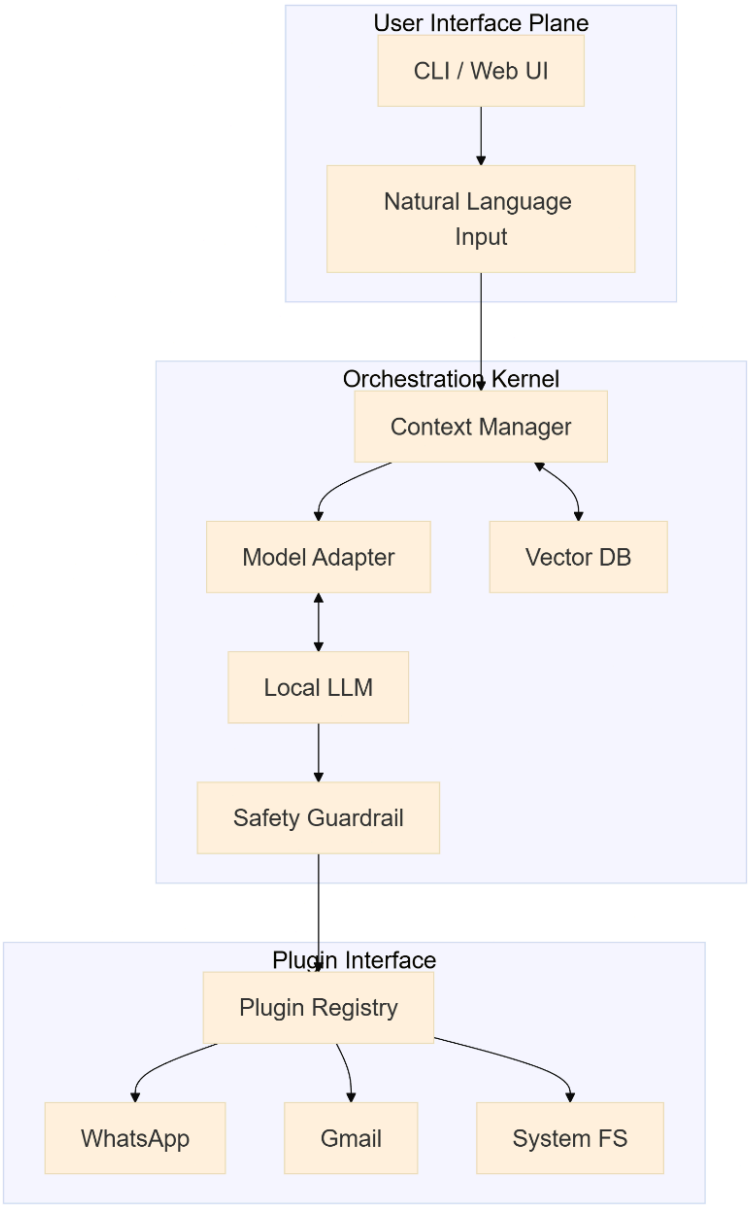
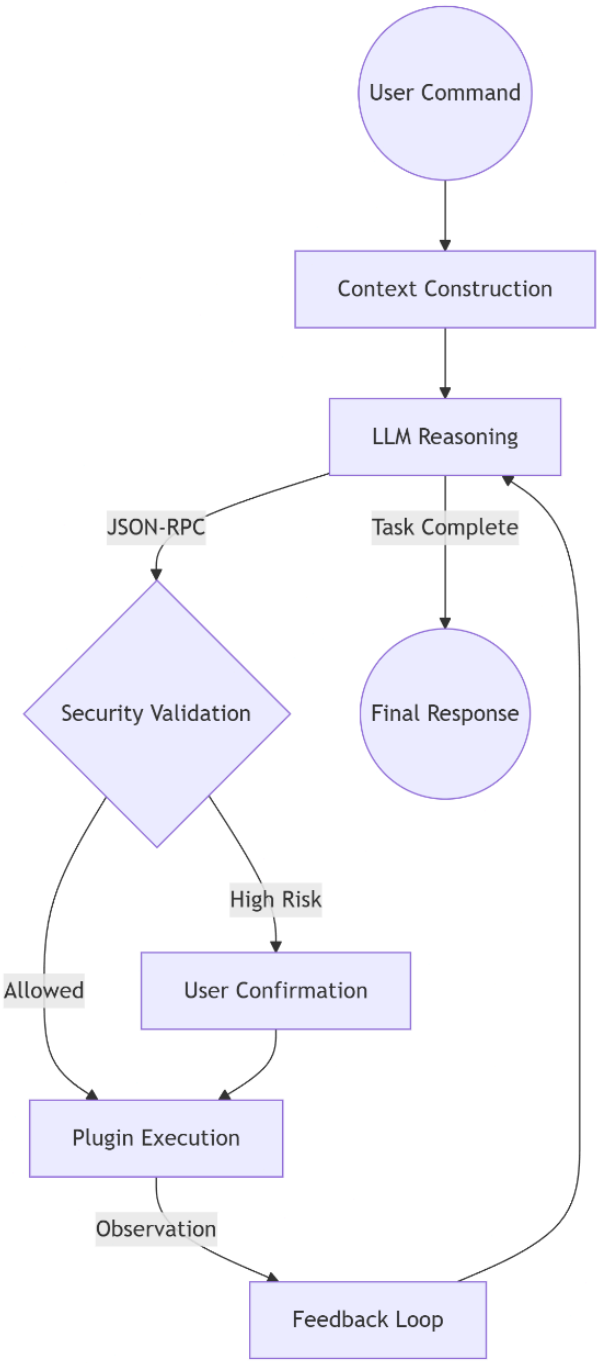


Diagram 2: The ReAct Loop (Cyclic Data Flow)

This diagram demonstrates the "Reason + Act" loop. The model does not just answer once; it enters a cycle of reasoning, executing, observing the result, and reasoning again until the task is complete.



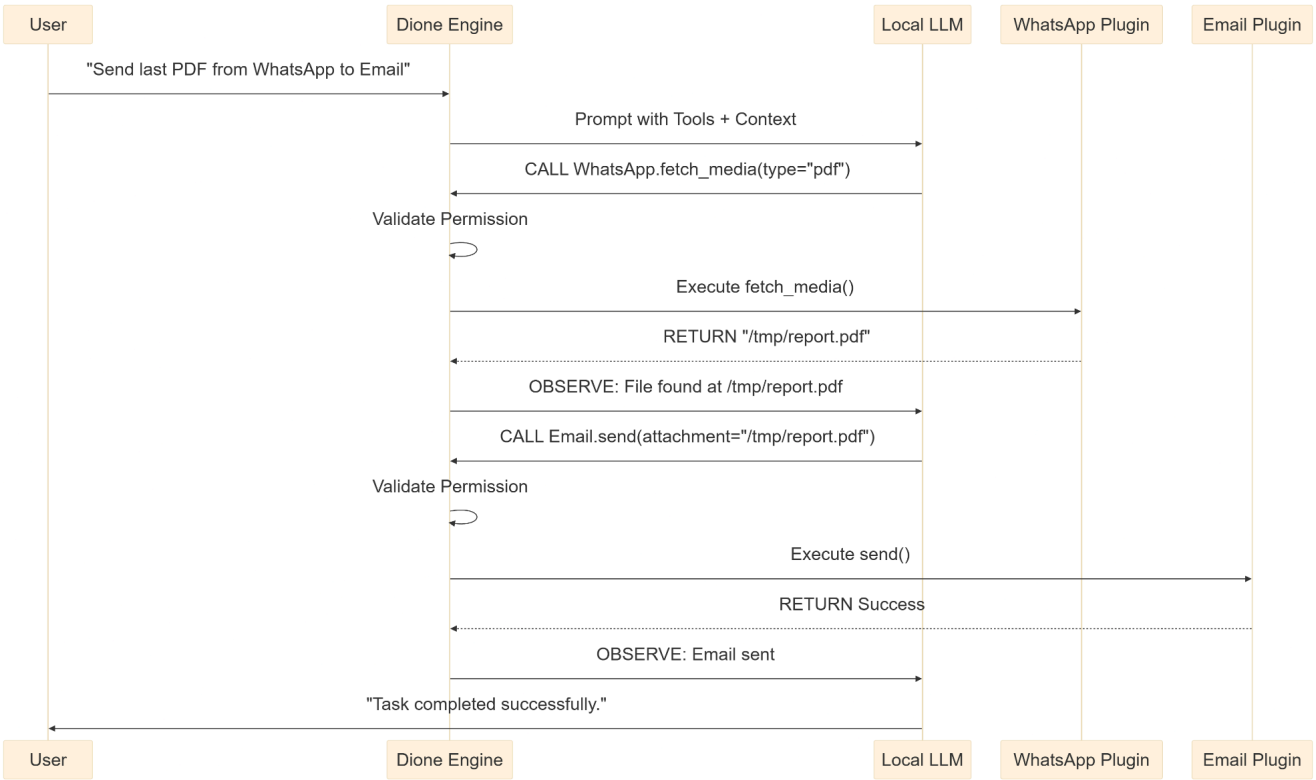
5.2 Operational Flow Description

The operational flow of Dione AI loops cyclically until a goal is achieved. This ensures that the model can correct its own errors and handle multi-step tasks dynamically.

1. **Input Layer & Intent Parsing:** The user interacts via a CLI or a local web interface. The input is not just text; it often includes "System State" (e.g., currently selected text, active window).
2. **Context Construction (RAG):** Before the LLM sees the request, the **Context Manager** retrieves relevant background information. It queries the local Vector Database (ChromaDB) for long-term memories (e.g., "The user prefers PDF summaries in bullet points") and appends the current tool schema (a list of available functions).
3. **Reasoning & Tool Selection (The "Brain"):** The Local LLM (e.g., Mistral-7B-Instruct) processes this enriched prompt. Instead of generating a conversational reply, the system prompt forces the model to output a structured **JSON-RPC** call.
   - *Input:* "Send the last PDF I received on WhatsApp to my email."
   - *Output:* `{"tool": "WhatsAppPlugin.fetch_media", "params": {"file_type": "pdf", "limit": 1}}`
4. **Validation & The Security Kernel:** Dione intercepts this JSON payload. It does not execute it immediately. It validates the schema (Are the parameters correct?) and checks the **Permission Manifest** (Is the user allowing WhatsApp access right now?). If the action is high-risk (e.g., deleting files), the system pauses for explicit user confirmation.

Diagram 3: Sequence of Events (Task Example)

A specific example of the flow when a user asks to "Send the last PDF from WhatsApp to Email."



5. **Execution (The "Engine"):** Once validated, the specific Python function in the `plugins/` directory is executed. This runs in a separate thread to prevent blocking the UI.
6. **Observation Feedback Loop:** The output of the function (e.g., `Success: Found 'Annual_Report.pdf' at /tmp/downloads/`) is not shown to the user yet. It is fed back into the LLM as an "Observation."
7. **Final Synthesis:** The LLM analyzes the observation. It realizes it has the file and generates the next step: `EmailPlugin.send(attachment='Annual_Report.pdf')`. This loop continues until the model determines the task is complete.

5.3 Core Components

- **The Context Manager:** This component acts as the "Hippocampus" of the system. It manages the sliding context window to ensure the model doesn't exceed its token limit (e.g., 4096 tokens). It intelligently summarizes older turns of the conversation and retrieves relevant "memories" from the vector store, ensuring the agent retains context over long sessions.
- **The Plugin Registry:** A dynamic loader that scans a local directory for Python scripts. It utilizes Python decorators (e.g., `@dione_tool`) to automatically parse function signatures and docstrings into a JSON schema that the LLM can understand. This allows developers to add new capabilities (like "Control Spotify" or "Read Notion") simply by dropping a script into a folder, without modifying the core engine.
- **The Model Adapter:** An abstraction layer that insulates the core logic from the specific inference backend. Whether the user is running `Ollama`, `Llama.cpp`, or `MLX` (on Apple Silicon), the Adapter provides a unified API surface. This ensures Dione remains compatible with the rapidly evolving landscape of open-source models.
- **The Safety Guardrail:** A distinct heuristic layer that scans model outputs for dangerous patterns (e.g., `rm -rf`, SQL injection attempts, or exfiltration of sensitive keys) and sanitizes inputs before they reach the plugin layer.

HARDWARE, SOFTWARE, AND TOOLS REQUIREMENTS

To ensure Dione AI is accessible to consumers while remaining capable of complex reasoning, the project targets a "High-End Consumer" hardware profile, with specific optimizations for Apple Silicon and NVIDIA architectures.

6.1 Software Requirements

- **Programming Language:** Python 3.10+ is selected as the primary language due to its unparalleled ecosystem for AI (PyTorch, LangChain) and system automation.
- **Local Inference Backend:**
  - **Ollama:** Used for its ease of deployment and REST API, acting as the server for the LLM.
  - **Llama.cpp:** Employed for raw GGUF model execution, allowing high-performance inference on CPU-only machines if necessary.
- **Orchestration Framework:** A custom implementation of the **LangChain** protocol is used to manage the "Chain of Thought" and tool-calling structures.
- **Database:**
  - **SQLite:** For structured logging of tasks, errors, and plugin configurations.
  - **ChromaDB:** A local, open-source vector store used to index user documents and conversation history for semantic retrieval.
- **Validation Libraries: Pydantic** is essential for enforcing strict type checking on LLM outputs, ensuring that the model does not hallucinate invalid parameters for function calls.
- **API Integrations:** Specific adapters for Google Workspace (Gmail/Calendar), Twilio (SMS), and OS-level accessibility APIs (AppleScript/PowerShell) to perform desktop actions.

6.2 Hardware Requirements

- **Processor (CPU):** Minimum Quad-core CPU (Intel i5 10th Gen or AMD Ryzen 5 3600). The CPU handles the orchestration logic, vector retrieval, and plugin execution.
- **System Memory (RAM):**
  - *Minimum:* 16GB. (8GB is reserved for the OS and browser, leaving ~8GB for a quantized 7B model).
  - *Recommended:* 32GB+. This allows for larger context windows and the simultaneous operation of the agent alongside heavy productivity apps.
- **Graphics Processing Unit (GPU) - The Critical Component:**
  - *NVIDIA:* RTX 3060 (12GB VRAM) is the baseline recommendation. 12GB is sufficient to load a 4-bit quantized Mistral-7B or Llama-3-8B model entirely into VRAM for fast token generation.
  - *Apple Silicon:* M1/M2/M3 chips with Unified Memory Architecture are highly recommended. Their ability to share high-bandwidth memory between CPU and GPU makes them exceptionally efficient for local LLMs.
- **Storage:** 50GB SSD space. High-speed NVMe storage is required to reduce model loading times and latency during vector database lookups.

PROPOSED EVALUATION MEASURES

The success of Dione AI cannot be measured by standard LLM benchmarks (like MMLU) alone, as those measure knowledge, not *agency*. Evaluation will focus on the system's ability to act correctly, safely, and efficiently.

7.1 Functional Accuracy (Success Rate)

- **Metric:** Task Success Rate (TSR).
- **Test Protocol:** A curated "AgentBench" suite of 50 multi-step tasks of increasing complexity.
  - *Level 1:* "What is the weather?" (Single tool).
  - *Level 2:* "Find the email from John and summarize it." (Search + Read + Summarize).
  - *Level 3:* "Check my calendar for conflicts, then email Sarah to reschedule." (Read + Reason + Write).
- **Goal:** Achieve >85% success rate. "Success" is defined as the system reaching the correct final state (e.g., the email was sent to the right person) without user intervention.
- **Failure Analysis:** Failures will be categorized into "Hallucination" (inventing non-existent tools), "Parameter Error" (sending a string instead of an int), or "Logic Error" (correct tools, wrong order).

7.2 System Latency (Responsiveness)

- **Metric:** Time-to-Action (TTA) and Token Generation Speed (t/s).
- **Test Protocol:** Measure the latency from the moment the user presses "Enter" to the moment the *first* tool is executed. This includes context retrieval time, LLM inference time, and JSON parsing time.
- **Goal:** TTA < 3 seconds on recommended hardware. This ensures the agent feels "responsive" rather than sluggish.

7.3 Privacy Validation

- **Metric:** Data Egress Volume & Destination Analysis.
- **Test Protocol:** The system will be subjected to a "Clean Room" test where network traffic is monitored via **Wireshark** and **Little Snitch**. The agent will be asked to process sensitive text (e.g., "Here is my password: XYZ").
- **Goal:** Zero unauthorized packets to external IPs. The only allowable traffic is to the specific API endpoints invoked by the user's requested tools (e.g., `api.openai.com` is forbidden; `imap.gmail.com` is allowed).

7.4 Safety & Stability

- **Metric:** Crash Rate and Harmful Action Rate.
- **Test Protocol:** Adversarial testing (Red Teaming) where the agent is given ambiguous or dangerous commands (e.g., "Delete all files that look old").
- **Goal:** The system must correctly trigger the "Safety Guardrail" and request user confirmation for any destructive action, achieving a 100% block rate for filesystem deletion commands that lack explicit user permission.

CONCLUSION

Dione AI represents a pivotal inflection point in the evolution of personal computing—one that is not merely incremental but structurally transformative. It formalizes the long-anticipated shift from *Intelligence as a Service* to *Intelligence as a Utility*, a model in which cognitive computation becomes as ubiquitous, infrastructural, and sovereign as electricity. In the current cloud-dominant landscape, users are effectively reduced to tenants of cognitive labor, compelled to externalize their most sensitive mental states—their uncertainties, creative impulses, financial vulnerabilities, strategic deliberations, and interpersonal reflections—to distant servers in exchange for algorithmic assistance. This dynamic has tacitly normalized the commodification of private cognition, creating what can only be described as a *Panopticon of Agency*: a regime where every query, preference, and hesitation is logged, cross-correlated, and potentially monetized.

Dione disrupts this architecture at its root. By demonstrating that complex, multi-stage agentic workflows can be executed entirely on local hardware—with no reliance on centralized inference endpoints—it collapses the presumed dichotomy between capability and privacy. It proves that the operational power of Large Action Models (LAMs) does not intrinsically require surveillance capitalism as its substrate. Instead, Dione establishes a new architectural doctrine: that users can possess both sovereign control over their data and access to high-throughput cognitive automation. This inversion is analogous in magnitude to the transition from mainframe computing to the personal computer. Just as the PC restored computational autonomy to individuals—liberating them from institutional dependency—Dione restores *agentic autonomy*, ensuring that the next generation of intelligent systems is distributed, local-first, and fundamentally user-owned.

This reorientation extends beyond mere privacy preservation. Dione functions as a *Digital Nervous System*—a persistent, context-rich substrate that binds intention to action with minimal friction. It operates not as an outsourced cognitive service with opaque allegiances but as a loyal prosthetic extension of its operator's will. Cloud-based agents remain structurally beholden to the corporate, legal, and reputational constraints of their providers; their ultimate loyalty is to the platform, not the user. Consequently, they routinely over-implement risk aversion, reject benign tasks, or distort outputs to conform to organizational policies. Dione, by residing entirely under the user's jurisdiction, aligns exclusively with the user's lawful intent. It becomes an instrument of digital self-determination rather than a mediator governed by third-party incentives.

Equally significant is Dione's open plugin architecture, which democratizes the evolution of its capabilities. Instead of innovation being funneled through the strategic priorities of centralized corporations, the ecosystem becomes community-driven, modular, and transparent. As consumer-grade accelerators continue to grow in throughput and energy efficiency, and as open-source models narrow the performance delta with their proprietary counterparts, Dione's local-first paradigm becomes not only viable but inevitable. High-trust applications—scientific research, legal reasoning, health data management, enterprise operations—will increasingly demand environments where neither context nor computation ever leaves the device.

Ultimately, Dione is far more than a tool for task automation; it is a constitutional statement for the future of human–machine interaction. It asserts that as synthetic intelligence becomes deeply integrated into our workflows, our cognition, and our daily decision-making, these systems must remain subordinate, transparent, and under our direct control. It stands as a bulwark against a trajectory where intelligent assistants risk becoming corporate intermediaries—entities that appear helpful but operate as double agents for the cloud. With Dione, the locus of power returns decisively to where it always belonged: the user. This is not merely a technical milestone; it is a declaration of digital independence.

REFERENCES

1. Schick, T., et al. (2023). "Toolformer: Language Models Can Teach Themselves to Use Tools." *arXiv preprint arXiv:2302.04761*.
2. Wei, J., et al. (2022). "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." *Advances in Neural Information Processing Systems*.
3. Nakajima, Y. (2023). "BabyAGI: Task-driven Autonomous Agent." *GitHub Repository*.
4. OpenAI. (2023). "GPT-4 Technical Report." *OpenAI*.
5. Chase, H. (2022). "LangChain: Building applications with LLMs through composability."
6. Yao, S., et al. (2023). "ReAct: Synergizing Reasoning and Acting in Language Models." *arXiv preprint arXiv:2210.03629*.
7. Touvron, H., et al. (2023). "Llama 2: Open Foundation and Fine-Tuned Chat Models." *Meta AI*.
8. Jiang, A. Q., et al. (2023). "Mistral 7B." *arXiv preprint arXiv:2310.06825*.
9. Lewis, P., et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Advances in Neural Information Processing Systems*.
10. Dettmers, T., et al. (2023). "QLoRA: Efficient Finetuning of Quantized LLMs." *arXiv preprint arXiv:2305.14314*.
11. Vaswani, A., et al. (2017). "Attention Is All You Need." *Advances in Neural Information Processing Systems*.
12. Park, J. S., et al. (2023). "Generative Agents: Interactive Simulacra of Human Behavior." *arXiv preprint arXiv:2304.03442*.
13. Mialon, G., et al. (2023). "Augmented Language Models: A Survey." *arXiv preprint arXiv:2302.07842*.
14. Significant-Gravitas. (2023). "AutoGPT: An Autonomous GPT-4 Experiment." *GitHub Repository*.
15. Kandpal, N., et al. (2023). "Large Language Models Struggle to Learn Long-Tail Knowledge." *arXiv preprint arXiv:2211.08411*.